Project 2 report - RailYard Dispatcher

The Romantic Tour group
Minh Duong Nguyen (U80322005) & Ngoc Viet Nguyen (U14177860)

Project description

RailYard Dispatcher is a C++17 console program that simulates a train yard. The user plays the dispatcher: build rail cars, couple them to the front or rear of the train, shunt cars onto a siding, run a safety inspection, and depart to earn revenue. The train is a std::deque of rail cars, so cars attach or detach from either end. The siding is built on the same base class and uses its deque as a stack.

The program makes sure a train follows real rules before it leaves: it needs an engine at the front, it can't be too heavy for its engines to pull, a dangerous tanker car can't sit right next to a passenger car, the train can't be too long, and it must carry something besides the engines. The program checks every rule before departure and tells the user about every rule that fails, not just the first one it finds.

Check the codebase at: GitHub
More about the project: README

Implementation explanation

Deque

The train is stored in a std::deque called cars_ (Track.h:23). A deque lets you add or remove items from the front or the back. The train uses both ends: push_front (Train.cpp:13) and push_back (Train.cpp:19) add a car to the front or back, and pop_front (Train.cpp:32) and pop_back (Train.cpp:39) remove a car from the front or back. front() (Train.cpp:31) and back() (Train.cpp:38) give access to the first and last car, which the program uses to take the car off either end. The siding inherits the same deque from Track, but only uses push_back (Siding.cpp:6) and pop_back (Siding.cpp:13), so it acts like a stack instead of a two-sided line.

Standard Library

The program uses several functions from the C++ standard library to work with the train.

- <iostream>: *cout* and *cin* for the menu and user input (DispatcherMenu.cpp:2)
- <string>: *string* for car descriptions (RailCar.h:4) , and *to_string* to turn numbers into text
- <deque>: *deque* to store the train and the siding (Track.h:4)

- <memory>: *unique_ptr* makes each car one owner (RailCar.h:32), *move* transfer a car between them without copying (Train.cpp:13, Siding.cpp:6)
- <algorithm>: *sort* orders car by weight (Train.cpp:88), *reverse* flips the order of the train (Train.cpp:92)
- *<cstdlib>: exit* ends the program when the input runs out (InputUtils.cpp:13)

Inheritance

RailCar is the base class (RailCar.h:11). Locomotive, PassengerCar, FreightCar, and TankerCar are all subclasses that inherit from RailCar. Each one calculates its own weight and how much money it earns in its own way, so the rest of the program does not need to know what kind of car it is looking at, it just calls the same function names. TankerCar also adds an extra feature on top: it marks itself as dangerous cargo when needed.

Track is also a base class (Track.h:10). Train and Siding both inherit from it. Train lets the user add or remove cars from either end. Siding only lets the user add or remove cars from one end, so it behaves like a stack even though it is built on the same kind of class.

Instructions:

Instructions to run

```
g++ -std=c++17 -Wall -Wextra -pedantic cars/*.cpp track/*.cpp app/*.cpp
main.cpp -o railyard
./railyard
```

Instructions to interact with program

The program runs a numbered menu. After every action it prints the train so the result is visible.

```
=== RailYard Dispatcher ===
 1. View train
 2. Couple car to front
 3. Couple car to rear
 4. Uncouple front car
 5. Uncouple rear car
 6. Shunt rear car to siding
 7. Pull car from siding
 8. Insert car at position
 9. Remove car at position
10. Find car by ID
11. Sort by weight
12. Reverse train
13. Statistics
14. Inspect and depart
15. Load demo train
 0. Exit
Choice:
```

Add cars: Option 2 couples a car to the front. Option 3 couples a car to the rear. Option 8 inserts a car at a chosen position.

Remove cars: Option 4 uncouples the front car. Option 5 uncouples the rear car. Option 9 removes a car at a chosen position. Option 6 shunts the rear car to the siding. Option 7 pulls a car off the siding back onto the train.

Other functions: Option 1 views the train. Option 10 finds a car by ID. Option 11 sorts by weight. Option 12 reverses the train. Option 13 shows statistics. Option 14 inspects and departs. Option 15 loads a demo train. Option 0 exits.

Sample:

```
Choice: 2                                (couple car to front)
Type (1 Locomotive, 2 Passenger, 3 Freight, 4 Tanker): 1

FRONT [L1] REAR
Siding (top is last): empty

Choice: 2
Type (1 Locomotive, 2 Passenger, 3 Freight, 4 Tanker): 4
Cargo tons: 67                           (too heavy for a tanker)
Please enter a number from 0 to 60.
Cargo tons: 60
Dangerous cargo? (y/n): y

FRONT [T2!]=[L1] REAR                    (tanker is in front of the locomotive)
Siding (top is last): empty

Choice: 12                               (reverse train)

FRONT [L1]=[T2!] REAR
Siding (top is last): empty

Choice: 14                               (inspect and depart)
Inspecting...
Inspection passed.

=== Departure report: trip 1 ===
  [#1] Locomotive | pulls 300.0 t | 60.0 t | revenue $0.0
  [#2] Tanker | 60.0 t cargo | 90.0 t | DANGEROUS | revenue $2250.0
  Total weight: 150.0 t
  Total revenue: $2250.0

Choice: 6                                (shunt rear car to siding)

FRONT [L1] REAR
Siding (top is last): [T2!]

Choice: 14                               (nothing to carry now)
Inspecting...
  Rule 5: the train has no cargo or passengers.
Inspection failed. The train cannot depart.
```

Roles

Minh Duong Nguyen (U80322005): 50%	Ngoc Viet Nguyen (U14177860): 50%
Implemented DispatcherMenu, InputUtils, and main.cpp, with all 15 menu options	Implemented the car classes and main.cpp skeleton and README draft
Tested the full flow with scripted input and finalized the README	Wrote the PDF report

Academic Integrity

"We understand that there will be no tolerance towards academic dishonesty, and that cheating will lead to an academic referral. We are aware of the identified behaviors that are considered violations of the academic standards for Undergraduate and Graduate students per USF policy."